

EX. NO. 10**TROUBLESHOOTING FABRIC APPLICATIONS****AIM**

To identify, analyze, and resolve common issues encountered in Hyperledger Fabric applications by troubleshooting network connectivity, chaincode deployment, channel configuration, certificate authentication, and Docker container errors.

SOFTWARE REQUIREMENTS

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images
- Visual Studio Code

HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

PROCEDURE**Step 1: Navigate to the Fabric Test Network**

Navigate to the Hyperledger Fabric test network directory.

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Hyperledger Fabric Network

Stop any existing network and start a fresh Fabric network with the application channel and Certificate Authorities.

./network.sh down

./network.sh up createChannel -ca

```
vboxuser@Ubuntu: ~/fabric-dev/fabric-samples/test-network$ cd ~/fabric-dev/fabric-samples/test-network
vboxuser@Ubuntu: ~/fabric-dev/fabric-samples/test-network$ ./network.sh up createChannel -ca
Using docker and docker compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' second
s and using database 'leveldb with crypto from 'Certificate Authorities'
Bringing up network
```

Step 3: Verify Docker Containers

Check whether all required Fabric containers are running. docker

ps

Verify the peer nodes, orderer, and Certificate Authority containers.

```
vboxuser@Ubuntu: ~/fabric-dev/fabric-samples/test-network$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        S
TATUS          PORTS
2c5baef255f3   hyperledger/fabric-peer:latest      "peer node start"       31 seconds ago U
p 30 seconds   0.0.0.0:9051->9051/tcp, [::]:9051->9051/tcp, 7051/tcp, 0.0.0.0:9445->9445/tcp,
[::]:9445->9445/tcp   peer0.org2.example.com
a44b18b6714e   hyperledger/fabric-orderer:latest   "orderer"               31 seconds ago U
p 30 seconds   0.0.0.0:7050->7050/tcp, [::]:7050->7050/tcp, 0.0.0.0:7053->7053/tcp, [::]:7053-
->7053/tcp, 0.0.0.0:9443->9443/tcp, [::]:9443->9443/tcp   orderer.example.com
ab1b97191d8a   hyperledger/fabric-peer:latest      "peer node start"       31 seconds ago U
p 30 seconds   0.0.0.0:7051->7051/tcp, [::]:7051->7051/tcp, 0.0.0.0:9444->9444/tcp, [::]:9444-
->9444/tcp           peer0.org1.example.com
14b030542e7e   hyperledger/fabric-ca:latest        "sh -c 'fabric-ca-se..." 45 seconds ago U
p 44 seconds   0.0.0.0:7054->7054/tcp, [::]:7054->7054/tcp, 0.0.0.0:17054->17054/tcp, [::]:170
54->17054/tcp       ca_org1
818d5fea0349   hyperledger/fabric-ca:latest        "sh -c 'fabric-ca-se..." 45 seconds ago U
p 44 seconds   0.0.0.0:9054->9054/tcp, [::]:9054->9054/tcp, 7054/tcp, 0.0.0.0:19054->19054/tcp
, [::]:19054->19054/tcp   ca_orderer
9d18e23bd759   hyperledger/fabric-ca:latest        "sh -c 'fabric-ca-se..." 45 seconds ago U
p 44 seconds   0.0.0.0:8054->8054/tcp, [::]:8054->8054/tcp, 7054/tcp, 0.0.0.0:18054->18054/tcp
, [::]:18054->18054/tcp   ca_org2
```

Step 4: Check Peer Logs

Display recent logs from the Org1 peer.

docker logs --tail 50 peer0.org1.example.com

Use the logs to identify peer startup, communication, channel, and chaincode-related problems.

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ docker logs --tail 50 peer0.org1.example.com
2026-08-31 16:48:12.384 UTC 001c INFO [discovery] NewService -> Created with config TLS: true, authCacheMaxSize: 1000, authCachePurgeRatio: 0.750000
2026-08-31 16:48:12.384 UTC 001d INFO [nodeCmd] serve -> Discovery service activated
2026-08-31 16:48:12.384 UTC 001e INFO [nodeCmd] serve -> Starting peer with Gateway enabled
2026-08-31 16:48:12.384 UTC 001f INFO [nodeCmd] serve -> Starting peer with ID=[peer0.org1.example.com], network ID=[dev], address=[peer0.org1.example.com:7051]
2026-08-31 16:48:12.386 UTC 0020 INFO [nodeCmd] serve -> Started peer with ID=[peer0.org1.example.com], network ID=[dev], address=[peer0.org1.example.com:7051]
2026-08-31 16:48:12.387 UTC 0021 INFO [kvledger] LoadPreResetHeight -> Loading prereset height from path [/var/hyperledger/production/ledgersData/chains]
2026-08-31 16:48:12.387 UTC 0022 INFO [blkstorage] preResetHtFiles -> No active channels passed
2026-08-31 16:48:18.742 UTC 0023 INFO [ledgermgmt] CreateLedger -> Creating ledger [mychannel] with genesis block
2026-08-31 16:48:18.783 UTC 0024 INFO [blkstorage] newBlockfileMgr -> Getting block information
```

Step 5: Check Orderer Logs

Display recent logs from the ordering service. docker

logs --tail 50 orderer.example.com

Use the logs to identify ordering-service and block-processing issues.

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ docker logs --tail 50 orderer.example.com
Version: v2.5.16
Commit SHA: f871cf9
Go version: go1.26.4
OS/Arch: linux/amd64
2026-08-31 16:48:11.755 UTC 000f INFO [orderer.common.server] Main -> Beginning to serve requests
2026-08-31 16:48:15.437 UTC 0010 INFO [blkstorage] newBlockfileMgr -> Getting block information from block storage
2026-08-31 16:48:15.466 UTC 0011 INFO [orderer.consensus.etcdraft] HandleChain -> EvictionSuspicion not set, defaulting to 10m0s
2026-08-31 16:48:15.466 UTC 0012 INFO [orderer.consensus.etcdraft] HandleChain -> Without system channel: after eviction Registrar.SwitchToFollower will be called
2026-08-31 16:48:15.467 UTC 0013 INFO [orderer.consensus.etcdraft] createOrReadWAL -> No WAL data found, creating new WAL at path '/var/hyperledger/production/orderer/etcdraft/wal/mychannel' channel=mychannel node=1
```

Step 6: Verify Channel Membership

Load the Org1 environment if required.

source setOrg1.sh

List the channels joined by the peer.

peer channel list

Verify that mychannel is available.

Step 7: Verify Installed Chaincode

Check the chaincode packages installed on the peer.

peer lifecycle chaincode queryinstalled

This helps determine whether the required chaincode package has been installed correctly.

Step 8: Troubleshoot Chaincode Deployment

If the chaincode has not been installed or committed correctly, redeploy it.

`./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/chaincodejavascript`

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer channel list
2026-08-31 16:51:48.475 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Channels peers has joined:
mychannel
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ source setOrg1.sh
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript
Using docker and docker compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
```

Step 9: Verify Chaincode Execution

Query the ledger to verify that the chaincode is functioning correctly. peer

`chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'`

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 --ordererTLSTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic --peerAddresses localhost:7051 --tlsRootCertFiles $ORG1_TLS --peerAddresses localhost:9051 --tlsRootCertFiles $ORG2_TLS -c '{"function":"InitLedger","Args":[]}'
2026-08-31 16:53:30.660 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"}, {"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Brad","Size":5,"docType":"asset"}, {"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"}, {"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"}, {"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}, {"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
```

Step 10: Check Docker Resource Usage

Monitor CPU and memory usage of the Fabric containers.

`docker stats`

Allow the statistics to run for a few seconds and press Ctrl+C to stop.

For a single resource-usage snapshot:

`docker stats --no-stream`

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ docker stats --no-stream
```

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O
2c5baef255f3	peer0.org2.example.com	7.26%	26.55MiB / 5.625GiB	0.46%	225kB / 154k
B	45.1kB / 393kB	9			
a44b18b6714e	orderer.example.com	0.39%	16.38MiB / 5.625GiB	0.28%	60.6kB / 207
kB	5.1MB / 193kB	8			
ab1b97191d8a	peer0.org1.example.com	8.96%	30.62MiB / 5.625GiB	0.53%	225kB / 154k
B	0B / 393kB	8			
14b030542e7e	ca_org1	0.00%	9.527MiB / 5.625GiB	0.17%	40.1kB / 50k
B	0B / 737kB	7			
818d5fea0349	ca_orderer	0.00%	11.05MiB / 5.625GiB	0.19%	60.5kB / 84.
6kB	999kB / 1.05MB	7			
9d18e23bd759	ca_org2	0.00%	9.633MiB / 5.625GiB	0.17%	37.9kB / 44k
B	1.16MB / 737kB	7			

Step 11: Restart the Fabric Network

If network components are not responding correctly, restart the Fabric network.

`./network.sh down`

Start the network again.

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh down
Using docker and docker compose
Stopping network
[+] down 1/6
: Container ca_org1 Stopping 1.3s
: Container ca_org2 Stopping 1.3s
: Container peer0.org1.example.com Stopping 1.3s
✓ Container orderer.example.com Removed 1.1s
: Container peer0.org2.example.com Stopping 1.3s
: Container ca_orderer Stopping 1.3s
```

`./network.sh up createChannel -ca` **Step 12:**

Verify the Network After Restart

Check the running containers.

`docker ps`

Verify channel membership.

`peer channel list`

Step 13: Clean Docker Resources Optionally

remove unused Docker resources. `docker`

`system prune -f`

This can be used to remove unused containers, networks, images, and other Docker resources.

COMMON ERRORS AND SOLUTIONS

Error	Possible Cause	Solution
Docker daemon not running	Docker service stopped	Start Docker using <code>sudo systemctl start docker</code>
Peer connection failed	Peer container stopped	Restart the Fabric network
Chaincode deployment failed	Incorrect chaincode path	Verify the -ccp directory
Channel not found	Channel was not created	Run <code>./network.sh createChannel</code>
Access denied	Invalid MSP or certificates	Verify the MSP configuration and certificates
Chaincode not installed	Installation failed	Redeploy the chaincode
TLS handshake failed	Incorrect TLS certificate Another service is using the port	Verify the TLS CA certificate paths Stop the conflicting process
Port already in use	the port	

TROUBLESHOOTING WORKFLOW

Start Fabric Network

|



Verify Docker Containers

|



Check Peer Logs

|



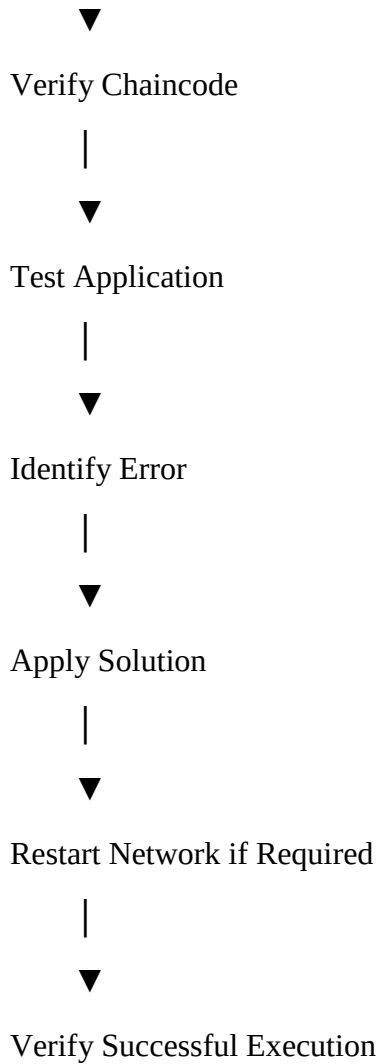
Check Orderer Logs

|



Verify Channel

|

**RESULT**

The Hyperledger Fabric application was successfully analyzed and common operational issues were identified and resolved using Docker commands, Fabric CLI tools, and log analysis. The experiment demonstrated effective troubleshooting techniques for network connectivity, chaincode deployment, channel configuration, certificate management, and resource monitoring.